Tensors and Einsum

Max Koch, Tamino Steinert

Tensors

Tensors

- Tensors are the basic data structure in many ML frameworks, e.g., PyTorch or Tensorflow
- Tensors are multidimensional arrays and can represent, e.g., scalars, vectors and matrices
- Tensor contractions can perform linear algebra operations, e.g., dot product, outer product and matrix multiplication
- Tensor contractions can be expressed with *einsum*

```python
import torch

# 0D tensor - scalar
s = torch.tensor(1)

# 1D tensor - vector
v = torch.tensor([1, 2])

# 2D tensor - matrix
m = torch.tensor([[1, 2],
                  [3, 4]])
```

Tensors

- Tensors are the basic data structure in many ML frameworks, e.g., PyTorch or Tensorflow
- Tensors are multidimensional arrays and can represent, e.g., scalars, vectors and matrices
- Tensor contractions can perform linear algebra operations, e.g., dot product, outer product and matrix multiplication
- Tensor contractions can be expressed with *einsum*

```python
import torch

# 0D tensor - scalar
s = torch.tensor(1)

# 1D tensor - vector
v = torch.tensor([1, 2])

# 2D tensor - matrix
m = torch.tensor([[1, 2],
                  [3, 4]])

# 3D tensor - "cube"
c = torch.tensor([[[1, 2],
                   [3, 4]],
                  [[5, 6],
                   [7, 8]]])

# ...
```

Tensors

- Tensors are the basic data structure in many ML frameworks, e.g., PyTorch or Tensorflow
- Tensors are multidimensional arrays and can represent, e.g., scalars, vectors and matrices
- Tensor contractions can perform linear algebra operations, e.g., dot product, outer product and matrix multiplication
- Tensor contractions can be expressed with *einsum*

```python
import torch

# 0D tensor - scalar
s = torch.tensor(1)

# 1D tensor - vector
v = torch.tensor([1, 2])

# 2D tensor - matrix
m = torch.tensor([[1, 2],
                  [3, 4]])

# 3D tensor - "cube"
c = torch.tensor([[[1, 2],
                   [3, 4]],
                  [[5, 6],
                   [7, 8]]])

# ...

# dot product
s2 = torch.dot(v, v)
# vector-matrix multiplication
v2 = torch.matmul(v, m)
# matrix-matrix multiplication
m2 = torch.matmul(m, m)
```

Tensors

- Tensors are the basic data structure in many ML frameworks, e.g., PyTorch or Tensorflow
- Tensors are multidimensional arrays and can represent, e.g., scalars, vectors and matrices
- Tensor contractions can perform linear algebra operations, e.g., dot product, outer product and matrix multiplication
- Tensor contractions can be expressed with *einsum*

```python
import torch

# 0D tensor - scalar
s = torch.tensor(1)

# 1D tensor - vector
v = torch.tensor([1, 2])

# 2D tensor - matrix
m = torch.tensor([[1, 2],
                  [3, 4]])

# 3D tensor - "cube"
c = torch.tensor([[[1, 2],
                   [3, 4]],
                  [[5, 6],
                   [7, 8]]])

# ...

# dot product
s2 = torch.dot(v, v)
# vector-matrix multiplication
v2 = torch.matmul(v, m)
# matrix-matrix multiplication
m2 = torch.matmul(m, m)

s3 = torch.einsum("k, k ->", v, v)
v3 = torch.einsum("k, kn -> n", v, m)
m3 = torch.einsum("mk, kn -> mn", m, m)
```

Tensors

- Tensors are the basic data structure in many ML frameworks, e.g., PyTorch or Tensorflow
- Tensors are multidimensional arrays and can represent, e.g., scalars, vectors and matrices
- Tensor contractions can perform linear algebra operations, e.g., dot product, outer product and matrix multiplication
- Tensor contractions can be expressed with *einsum*

```python
import torch

# 0D tensor - scalar
s = torch.tensor(1)

# 1D tensor - vector
v = torch.tensor([1, 2])

# 2D tensor - matrix
m = torch.tensor([[1, 2],
                  [3, 4]])

# 3D tensor - "cube"
c = torch.tensor([[[1, 2],
                   [3, 4]],
                  [[5, 6],
                   [7, 8]]])

# ...

# dot product
s2 = torch.dot(v, v)
# vector-matrix multiplication
v2 = torch.matmul(v, m)
# matrix-matrix multiplication
m2 = torch.matmul(m, m)

s3 = torch.einsum("k, k ->", v, v)
v3 = torch.einsum("k, kn -> n", v, m)
m3 = torch.einsum("mk, kn -> mn", m, m)
```

Mapping Tensors to Memory

- Tensors are multidimensional arrays
- Elements can be accessed through indices
- Memory / address space is linear — no concept of higher dimensions
- The mapping between memory and tensor indices is determined by its *strides*
- The stride of a dimension specifies how many elements to jump to access the next element along that dimension
- We assume general row-major format in this course — dimensions have descending strides, i.e., the last dimension has stride 1

```python
import torch
# 3D tensor where each dimension size is two
t = torch.tensor([[[1, 2],
                   [3, 4]],
                  [[5, 6],
                   [7, 8]]], dtype=torch.int8)

print(t.shape) # torch.Size([2, 2, 2])

print(t[0, 0, 0]) # tensor(1)
print(t[0, 0, 1]) # tensor(2)
print(t[0, 1, 0]) # tensor(3)
print(t[1, 0, 0]) # tensor(5)
print(t[1, 1, 1]) # tensor(8)
```

Mapping Tensors to Memory

- Tensors are multidimensional arrays
- Elements can be accessed through indices
- Memory / address space is linear — no concept of higher dimensions
- The mapping between memory and tensor indices is determined by its *strides*
- The stride of a dimension specifies how many elements to jump to access the next element along that dimension
- We assume general row-major format in this course — dimensions have descending strides, i.e., the last dimension has stride 1

```python
import torch
# 3D tensor where each dimension size is two
t = torch.tensor([[[1, 2],
                   [3, 4]],
                  [[5, 6],
                   [7, 8]]], dtype=torch.int8)

print(t.shape) # torch.Size([2, 2, 2])

print(t[0, 0, 0]) # tensor(1)
print(t[0, 0, 1]) # tensor(2)
print(t[0, 1, 0]) # tensor(3)
print(t[1, 0, 0]) # tensor(5)
print(t[1, 1, 1]) # tensor(8)

print(t.untyped_storage())
# 1, 2, 3, 4, 5, 6, 7, 8
# [torch.storage.UntypedStorage(device=cpu) of size 8]

print(t.stride()) # (4, 2, 1)
```

Mapping Tensors to Memory

- Tensors are multidimensional arrays
- Elements can be accessed through indices
- Memory / address space is linear — no concept of higher dimensions
- The mapping between memory and tensor indices is determined by its *strides*
- The stride of a dimension specifies how many elements to jump to access the next element along that dimension
- We assume general row-major format in this course — dimensions have descending strides, i.e., the last dimension has stride 1

```python
import torch
# 3D tensor where each dimension size is two
t = torch.tensor([[[1, 2],
                   [3, 4]],
                  [[5, 6],
                   [7, 8]]], dtype=torch.int8)

print(t.shape) # torch.Size([2, 2, 2])

print(t[0, 0, 0]) # tensor(1)
print(t[0, 0, 1]) # tensor(2)
print(t[0, 1, 0]) # tensor(3)
print(t[1, 0, 0]) # tensor(5)
print(t[1, 1, 1]) # tensor(8)

print(t.untyped_storage())
# 1, 2, 3, 4, 5, 6, 7, 8
# [torch.storage.UntypedStorage(device=cpu) of size 8]

print(t.stride()) # (4, 2, 1)

mem = bytearray(range(0, 8))
t_mem = torch.frombuffer(mem, dtype=torch.int8)
print(t_mem) # tensor([1, 2, 3, 4, 5, 6, 7, 8])

print(t_mem.view(2, 2, 2)) # tensor([[[1, 2],
                           #          [3, 4]],
                           #         [[5, 6],
                           #          [7, 8]]])
```

Mapping Tensors to Memory

- Tensors are multidimensional arrays
- Elements can be accessed through indices
- Memory / address space is linear — no concept of higher dimensions
- The mapping between memory and tensor indices is determined by its *strides*
- The stride of a dimension specifies how many elements to jump to access the next element along that dimension
- We assume general row-major format in this course — dimensions have descending strides, i.e., the last dimension has stride 1

```python
import torch
# 3D tensor where each dimension size is two
t = torch.tensor([[[1, 2],
                   [3, 4]],
                  [[5, 6],
                   [7, 8]]], dtype=torch.int8)

print(t.shape) # torch.Size([2, 2, 2])

print(t[0, 0, 0]) # tensor(1)
print(t[0, 0, 1]) # tensor(2)
print(t[0, 1, 0]) # tensor(3)
print(t[1, 0, 0]) # tensor(5)
print(t[1, 1, 1]) # tensor(8)

print(t.untyped_storage())
# 1, 2, 3, 4, 5, 6, 7, 8
# [torch.storage.UntypedStorage(device=cpu) of size 8]

print(t.stride()) # (4, 2, 1)

mem = bytearray(range(0, 8))
t_mem = torch.frombuffer(mem, dtype=torch.int8)
print(t_mem) # tensor([1, 2, 3, 4, 5, 6, 7, 8])

print(t_mem.view(2, 2, 2)) # tensor([[[1, 2],
                           #          [3, 4]],
                           #         [[5, 6],
                           #          [7, 8]]])
```

Tensor View

- A tensor view is defined by its shape and strides
- The same memory can be interpreted as different tensors
- A *view* of a tensor is a different interpretation of the same memory

```python
import torch

t = torch.tensor([[[1, 2],
                   [3, 4]],
                  [[5, 6],
                   [7, 8]]], dtype=torch.int8)
print(t.shape) # torch.Size([2, 2, 2])
print(t.stride()) # (4, 2, 1)
```

Tensor View

- A tensor view is defined by its shape and strides
- The same memory can be interpreted as different tensors
- A *view* of a tensor is a different interpretation of the same memory

```python
import torch

t = torch.tensor([[[1, 2],
                   [3, 4]],
                  [[5, 6],
                   [7, 8]]], dtype=torch.int8)
print(t.shape) # torch.Size([2, 2, 2])
print(t.stride()) # (4, 2, 1)


# view of 3D tensor as 1D tensor
# tensor([1, 2, 3, 4, 5, 6, 7, 8])
t_1D = t.view(8)
print(t_1D.shape) # torch.Size([8])
print(t_1D.stride()) # (1,)
```

Tensor View

- A tensor view is defined by its shape and strides
- The same memory can be interpreted as different tensors
- A *view* of a tensor is a different interpretation of the same memory

```python
import torch

t = torch.tensor([[[1, 2],
                   [3, 4]],
                  [[5, 6],
                   [7, 8]]], dtype=torch.int8)
print(t.shape) # torch.Size([2, 2, 2])
print(t.stride()) # (4, 2, 1)


# view of 3D tensor as 1D tensor
# tensor([1, 2, 3, 4, 5, 6, 7, 8])
t_1D = t.view(8)
print(t_1D.shape) # torch.Size([8])
print(t_1D.stride()) # (1,)


# view of 3D tensor as 2D tensor
# tensor([[1, 2, 3, 4],
#         [5, 6, 7, 8]])
t_2D = t.view(2, 4)
print(t_2D.shape) # torch.Size([2, 4])
print(t_2D.stride()) # (4, 1)
```

Tensor View

- A tensor view is defined by its shape and strides
- The same memory can be interpreted as different tensors
- A *view* of a tensor is a different interpretation of the same memory

```python
import torch

t = torch.tensor([[[1, 2],
                   [3, 4]],
                  [[5, 6],
                   [7, 8]]], dtype=torch.int8)
print(t.shape) # torch.Size([2, 2, 2])
print(t.stride()) # (4, 2, 1)


# view of 3D tensor as 1D tensor
# tensor([1, 2, 3, 4, 5, 6, 7, 8])
t_1D = t.view(8)
print(t_1D.shape) # torch.Size([8])
print(t_1D.stride()) # (1,)


# view of 3D tensor as 2D tensor
# tensor([[1, 2, 3, 4],
#         [5, 6, 7, 8]])
t_2D = t.view(2, 4)
print(t_2D.shape) # torch.Size([2, 4])
print(t_2D.stride()) # (4, 1)


# all three tensors share the same memory
print(t.data_ptr() == t_1D.data_ptr() == t_2D.data_ptr())
# True
```

Tensor View

- A tensor view is defined by its shape and strides
- The same memory can be interpreted as different tensors
- A *view* of a tensor is a different interpretation of the same memory

Example

- What is the shape of `t_mystery = t.view(4, 2)`?
- What are the strides of `t_mystery`?
- What is the output of `print(t_mystery)`?

```python
import torch

t = torch.tensor([[[1, 2],
                   [3, 4]],
                  [[5, 6],
                   [7, 8]]], dtype=torch.int8)
print(t.shape) # torch.Size([2, 2, 2])
print(t.stride()) # (4, 2, 1)


# view of 3D tensor as 1D tensor
# tensor([1, 2, 3, 4, 5, 6, 7, 8])
t_1D = t.view(8)
print(t_1D.shape) # torch.Size([8])
print(t_1D.stride()) # (1,)


# view of 3D tensor as 2D tensor
# tensor([[1, 2, 3, 4],
#         [5, 6, 7, 8]])
t_2D = t.view(2, 4)
print(t_2D.shape) # torch.Size([2, 4])
print(t_2D.stride()) # (4, 1)


# all three tensors share the same memory
print(t.data_ptr() == t_1D.data_ptr() == t_2D.data_ptr())
# True
```

Tensor View

- A tensor view is defined by its shape and strides
- The same memory can be interpreted as different tensors
- A *view* of a tensor is a different interpretation of the same memory

Example

- What is the shape of `t_mystery = t.view(4, 2)`? `torch.Size([4, 2])`
- What are the strides of `t_mystery`?
- What is the output of `print(t_mystery)`?

```python
import torch

t = torch.tensor([[[1, 2],
                   [3, 4]],
                  [[5, 6],
                   [7, 8]]], dtype=torch.int8)
print(t.shape) # torch.Size([2, 2, 2])
print(t.stride()) # (4, 2, 1)


# view of 3D tensor as 1D tensor
# tensor([1, 2, 3, 4, 5, 6, 7, 8])
t_1D = t.view(8)
print(t_1D.shape) # torch.Size([8])
print(t_1D.stride()) # (1,)


# view of 3D tensor as 2D tensor
# tensor([[1, 2, 3, 4],
#         [5, 6, 7, 8]])
t_2D = t.view(2, 4)
print(t_2D.shape) # torch.Size([2, 4])
print(t_2D.stride()) # (4, 1)


# all three tensors share the same memory
print(t.data_ptr() == t_1D.data_ptr() == t_2D.data_ptr())
# True
```

Tensor View

- A tensor view is defined by its shape and strides
- The same memory can be interpreted as different tensors
- A *view* of a tensor is a different interpretation of the same memory

Example

- What is the shape of `t_mystery = t.view(4, 2)`? `torch.Size([4, 2])`
- What are the strides of `t_mystery`? `(2, 1)`
- What is the output of `print(t_mystery)`?

```python
import torch

t = torch.tensor([[[1, 2],
                   [3, 4]],
                  [[5, 6],
                   [7, 8]]], dtype=torch.int8)
print(t.shape) # torch.Size([2, 2, 2])
print(t.stride()) # (4, 2, 1)


# view of 3D tensor as 1D tensor
# tensor([1, 2, 3, 4, 5, 6, 7, 8])
t_1D = t.view(8)
print(t_1D.shape) # torch.Size([8])
print(t_1D.stride()) # (1,)


# view of 3D tensor as 2D tensor
# tensor([[1, 2, 3, 4],
#         [5, 6, 7, 8]])
t_2D = t.view(2, 4)
print(t_2D.shape) # torch.Size([2, 4])
print(t_2D.stride()) # (4, 1)


# all three tensors share the same memory
print(t.data_ptr() == t_1D.data_ptr() == t_2D.data_ptr())
# True
```

Tensor View

- A tensor view is defined by its shape and strides
- The same memory can be interpreted as different tensors
- A *view* of a tensor is a different interpretation of the same memory

Example

- What is the shape of `t_mystery = t.view(4, 2)`? `torch.Size([4, 2])`
- What are the strides of `t_mystery`? `(2, 1)`
- What is the output of `print(t_mystery)`? `tensor([[1, 2], [3, 4], [5, 6], [7, 8]])`

```python
import torch

t = torch.tensor([[[1, 2],
                   [3, 4]],
                  [[5, 6],
                   [7, 8]]], dtype=torch.int8)
print(t.shape) # torch.Size([2, 2, 2])
print(t.stride()) # (4, 2, 1)


# view of 3D tensor as 1D tensor
# tensor([1, 2, 3, 4, 5, 6, 7, 8])
t_1D = t.view(8)
print(t_1D.shape) # torch.Size([8])
print(t_1D.stride()) # (1,)


# view of 3D tensor as 2D tensor
# tensor([[1, 2, 3, 4],
#         [5, 6, 7, 8]])
t_2D = t.view(2, 4)
print(t_2D.shape) # torch.Size([2, 4])
print(t_2D.stride()) # (4, 1)


# all three tensors share the same memory
print(t.data_ptr() == t_1D.data_ptr() == t_2D.data_ptr())
# True
```

Tensor Slicing and Striding

- Slicing and striding can be used to create views of tensors without copying memory
- Syntax: `tensor[start:stop:step, ...]`
  - `start` (default: 0) is the index to start from (inclusive)
  - `stop` (default: size of dimension) is the index to stop at (exclusive)
  - `step` (default: 1) is the step size to take between indices
  - `:` includes the whole dimension

```python
import torch

t = torch.tensor([[[1, 2],
                   [3, 4]],
                  [[5, 6],
                   [7, 8]]], dtype=torch.int8)

# tensor([[1, 2, 3, 4],
#         [5, 6, 7, 8]])
t_2D = t.view(2, 4)

# tensor([[1, 3],
#         [5, 7]])
t_sliced = t_2D[0:2:1, 0:4:2]
t_sliced = t_2D[:, ::2] # same as above

# no unit stride, i.e., no stride-1 dimension
print(t_sliced.stride()) # (4, 2)
print(t_sliced.is_contiguous()) # False


# slice 3D tensor into two 2D tensors
m0 = t[0, :, :] # tensor([[1, 2],
                #         [3, 4]])

m1 = t[1, :, :] # tensor([[5, 6],
                #         [7, 8]])
```

Example

`print(t[0, 0, :])` `# tensor([1, 2])`

Einsum

- An *einsum* describes multidimensional summation over index products
- Example: matrix multiplication ($C = A \cdot B$):
  - $c_{ik} = \sum_j a_{ij} \cdot b_{jk}$
  - `einsum(ij, jk -> ik, A, B)`
- *einsum* syntax:
  - Each dimension is represented by a label, e.g., `i`, `j`, `k`
  - Tensors are represented by their dimension labels, e.g., `ij` for a 2D tensor
  - Input tensors are comma-separated, e.g., `ij, jk`
  - Labels of the output tensor are specified after `->`, e.g., `ij, jk -> ik`
  - For shared dimensions, the same label is used, e.g., a matrix multiplication has three unique dimensions
  - Input elements along corresponding dimensions are multiplied, e.g., $a_{ij} * b_{jk}$
  - If a dimension label only appears in the input tensors, it is summed over, e.g., $\sum_j$

Einsum Examples

Example

$c_{abc} = \sum_{jk} a_{abjk} \cdot b_{cjk}$

```
C = torch.einsum(abjk, cjk -> abc, A, B)
```

Einsum Examples

Example

$c_{abc} = \sum_{jk} a_{abjk} \cdot b_{cjk}$

```
C = torch.einsum(abjk, cjk -> abc, A, B)
```

$c_{bac} = \sum_{jk} a_{jbka} \cdot b_{kcj}$

Einsum Examples

Example

$c_{abc} = \sum_{jk} a_{abjk} \cdot b_{cjk}$

```
C = torch.einsum(abjk, cjk -> abc, A, B)
```

$c_{bac} = \sum_{jk} a_{jbka} \cdot b_{kcj}$

```
C = torch.einsum(jbka, kcj -> bac, A, B)
```

Einsum Examples

Example

$c_{abc} = \sum_{jk} a_{abjk} \cdot b_{cjk}$

```
C = torch.einsum(abjk, cjk -> abc, A, B)
```

$c_{bac} = \sum_{jk} a_{jbka} \cdot b_{kcj}$

```
C = torch.einsum(jbka, kcj -> bac, A, B)
```

$d_{xy} = \sum_{k} a_{xk} \cdot b_{yk} \cdot c_{xy}$

Einsum Examples

Example

$c_{abc} = \sum_{jk} a_{abjk} \cdot b_{cjk}$

```
C = torch.einsum(abjk, cjk -> abc, A, B)
```

$c_{bac} = \sum_{jk} a_{jbka} \cdot b_{kcj}$

```
C = torch.einsum(jbka, kcj -> bac, A, B)
```

$d_{xy} = \sum_{k} a_{xk} \cdot b_{yk} \cdot c_{xy}$

```
D = torch.einsum(xk, yk, xy -> xy, A, B, C)
```

Einsum Examples

Example

$c_{abc} = \sum_{jk} a_{abjk} \cdot b_{cjk}$

```
C = torch.einsum(abjk, cjk -> abc, A, B)
```

$c_{bac} = \sum_{jk} a_{jbka} \cdot b_{kcj}$

```
C = torch.einsum(jbka, kcj -> bac, A, B)
```

$d_{xy} = \sum_{k} a_{xk} \cdot b_{yk} \cdot c_{xy}$

```
D = torch.einsum(xk, yk, xy -> xy, A, B, C)
```

```
D = torch.einsum(xk, yk, zk -> xyz, A, B, C)
```

Einsum Examples

Example

$c_{abc} = \sum_{jk} a_{abjk} \cdot b_{cjk}$

```
C = torch.einsum(abjk, cjk -> abc, A, B)
```

$c_{bac} = \sum_{jk} a_{jbka} \cdot b_{kcj}$

```
C = torch.einsum(jbka, kcj -> bac, A, B)
```

$d_{xy} = \sum_{k} a_{xk} \cdot b_{yk} \cdot c_{xy}$

```
D = torch.einsum(xk, yk, xy -> xy, A, B, C)
```

$d_{xyz} = \sum_{k} a_{xk} \cdot b_{yk} \cdot c_{zk}$

```
D = torch.einsum(xk, yk, zk -> xyz, A, B, C)
```

Einsum Examples

Operation	Mathematical	Summation	*einsum*
Matrix product	$A \cdot B$	$c_{pr} = \sum_q a_{pq} b_{qr}$	`#('pq,qr->pr', A, B)`
Transposed matrix product	$A^T \cdot B$	$c_{pr} = \sum_q a_{qp} b_{qr}$	`#('qp,qr->pr', A, B)`
Hadamard product	$A \odot B$	$c_{pq} = a_{pq} b_{pq}$	`#('pq,pq->pq', A, B)`
sq'd Frobenius norm	$\lVert A \rVert_F^2$	$c = \sum_p \sum_q a_{pq} a_{pq}$	`#('pq,pq->', A, A)`
Khatri-Rao product	$A \circledcirc B$	$c_{pqr} = a_{pq} b_{qr}$	`#('pq,qr->pqr', A, B)`
Kronecker product	$A \otimes B$	$c_{pqrs} = a_{pq} b_{rs}$	`#('pq,rs->pqrs', A, B)`